

Homework6

March 4, 2019

```
In [5]: ! pip install xgboost
```

Collecting xgboost

Using cached <https://files.pythonhosted.org/packages/4f/4c/4969b10939c4557ae46e5569d07c0c7ce/>

Requirement already satisfied: numpy in /Users/rachel/anaconda3/lib/python3.7/site-packages (f

Requirement already satisfied: scipy in /Users/rachel/anaconda3/lib/python3.7/site-packages (f

Building wheels for collected packages: xgboost

Running setup.py bdist_wheel for xgboost ... done

Stored in directory: /Users/rachel/Library/Caches/pip/wheels/8a/89/0c/4870bb69132698f40889fa

Successfully built xgboost

Installing collected packages: xgboost

Successfully installed xgboost-0.81

```
In [7]: import numpy as np
import pandas as pd
# ! pip install xgboost
import xgboost
import xgboost as xgb
from pdpbox import pdp
from matplotlib import cm
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor, RandomForestClassifier
from sklearn.model_selection import GridSearchCV, RandomizedSearchCV
from sklearn.linear_model import LogisticRegression, LogisticRegressionCV
from sklearn.metrics import mean_absolute_error
import scipy
from xgboost import XGBRegressor
from pygam import LogisticGAM, s, f
```

```
In [8]: import warnings
warnings.filterwarnings("ignore")
```

```
In [10]: import matplotlib
matplotlib.rcParams['figure.figsize'] = (9,6)
```

1 Problem 1

1.1 (a)

```
In [ ]: #This function finds the best single split on one of the two variables, while incorporating
# Inputs:
## X: a numpy array with 2 columns
## y: a numpy array with n elements
## wgts: a numpy array of n observation weights.
# Returns two things:
## fit: The object returned by DecisionTreeClassifier
## tree_info: A dictionary with the following additional info. You don't need this, but
##   variable_idx: Number of selected variable column
##   threshold: Where that variable was split
##   guess_less: which category is guessed if less than threshold
##   guess_greater: which category is guessed if greater than threshold
def find_split(X, y, wgts=None):
    n = X.shape[0]
    dtc = DecisionTreeClassifier(max_depth=1)
    fit = dtc.fit(X,y, sample_weight=wgts)
    tree_info = {
        'variable_idx' : fit.tree_.feature[0],
        'threshold' : fit.tree_.threshold[0],
        'guess_less' : np.argmax(fit.tree_.value[1])*2-1,
        'guess_greater' : np.argmax(fit.tree_.value[2])*2-1
    }
    return(fit, tree_info)

#Draws the data points and the decision boundaries used by the boosted trees.
#If score is supplied, the points are also shaded by the score.
# Inputs:
## trees: the list of tree objects output by my_adaboost
## X: a set of points output by get_circle_data (no y is needed)
## score: n-vector of scores to color each point (optional)
# Outputs: makes a plot. You still need to call plt.show()
def draw_boosted_trees(trees, X, score=None):
    if score is not None:
        plt.scatter(X[:,0], X[:,1], c=score, cmap=plt.get_cmap('plasma'))
    else:
        plt.scatter(X[:,0], X[:,1])
    line_col='grey'
    for fit in trees:
        variable_idx = fit.tree_.feature[0]
        threshold = fit.tree_.threshold[0]
        if variable_idx==0:
            plt.axvline(x = threshold, color=line_col)
        else:
```

```

plt.axhline(y = threshold, color=line_col)

# Generates data for our example
# Inputs:
## n: number of samples
# Output: list with entries:
## X: an nx2 matrix of data points.
## y: class for the point (-1,1)
def get_circle_data(n):
    gaussian_2d = scipy.stats.multivariate_normal(mean = np.zeros(2), cov = np.eye(2))
    X = gaussian_2d.rvs(n)
    Y = np.where(X[:,0]**2+X[:,1]**2<1, 1, -1)
    return(X,Y)

#### YOU DON'T NEED TO CHANGE FUNCTIONS ABOVE THIS LINE, THEY ARE FREE ###

# Implement the adaboost function in my_adaboost
# Inputs:
## X, y: Data produced by get_circle_data()
## B: The number of trees to produce during boosting
# Returns:
## trees: a list of fitted tree objects
## alphas: a numpy array of B alpha values for the trees

def my_adaboost(X, y, B=10):
    n = X.shape[0]
    #Observation weights
    wgts = np.repeat(1/n, n)
    trees = [] #To store your trees. Use append()
    alphas = np.zeros(B) #To store your alphas
    for b in range(B):
        #Find the next split, according to your current weights.
        #Hint: the returned tree supports predict()
        tree, info = find_split(X, y, wgts)

        #TODO: Fill in the rest of the body of this loop to implement
        # the AdaBoost algorithm from class
        yhat= tree.predict(X)
        eb = np.sum(((yhat!=y)*wgts))/np.sum(wgts)

        #You can introduce whatever you need here. At the very least,
        #it needs to result in updated wgts and
        #an additional entry in the trees list and the alphas vector.

```

```

        #TODO: You'll probably need some stuff here#
        alphas[b] = np.log((1-eb)/eb) #Add the new tree's weight#
        trees.append(tree) #Store the new tree#
        wgts = wgts * np.exp(alphas[b]*(yhat!=y)) #Update your observation weights#

    return(trees, alphas)

#Calculate predictions for boosted trees from my_adaboost. If passed true y values, a
# Inputs:
## trees: list of trees returned by my_adaboost
## alphas: alpha vector returned by my_adaboost
## X: a data set to make predictions on
## y: the true labels for that data set, in {-1,1}. If given, running prediction error
# Outputs: a list containing:
## score: the score for each point (before thresholding)
## preds: the thresholded scores: our guesses at class.
## test_err: a length B vector of misclassification error at each step (1 tree to B tr

def predict_ada(trees, alphas, X, y=None):
    n = X.shape[0]
    #Initialize a score vector to zero.
    #This will store the ongoing sum alpha_b fhat_b(x) for each point.
    score = np.zeros(n)
    B = len(trees)
    #This will store the misclassification error at each step,
    # so we can see how it changes with the number of trees
    test_err = np.zeros(B)
    for b in range(B):
        #Update your score to include the contribution from tree b
        score = score + alphas[b]*trees[b].predict(X)
        preds = np.where(score>0, 1, -1)
        if y is not None:
            #Record the misclassification error for the predictions
            #based on thresholding the current score
            test_err[b] = np.mean(preds!=y)
    if y is None:
        return(score, preds)
    return(score, preds, test_err)

#Calculates the partial dependence plot over the first column of X
#Fixes the first column at each point in the grid and evaluates the
#predictions at all of these fake points. These predictions are used
# to obtain the partial dependence over x1.
# Inputs:
## trees: list of trees returned by my_adaboost
## alphas: list of alphas returned by my_adaboost

```

```

## X: data set to use for computing partial dependence.

def calculate_partial_dependence_x1(trees, alphas, X):
    m = 50#Number of grid points to evaluate partial dependence
    x = np.linspace(np.min(X[:,0]), np.max(X[:,0]), m)

    #Somewhere to store the partial dependence values
    pdep = np.zeros(m)

    #We will fake a dataset and change its x1 values to match each evaluation points
    fake = X.copy()
    for i in range(m):
        fake[:,0] = x[i]
        #TODO: Fill in the actual calculation of the partial
        # dependence at x[i] below#
        score = predict_ada(trees, alphas, fake)[1]
        #HINT: Evaluate predict_ada on the fake data set and use
        # resulting predictions
        pdep[i] = np.mean(score)
    return(x, pdep)

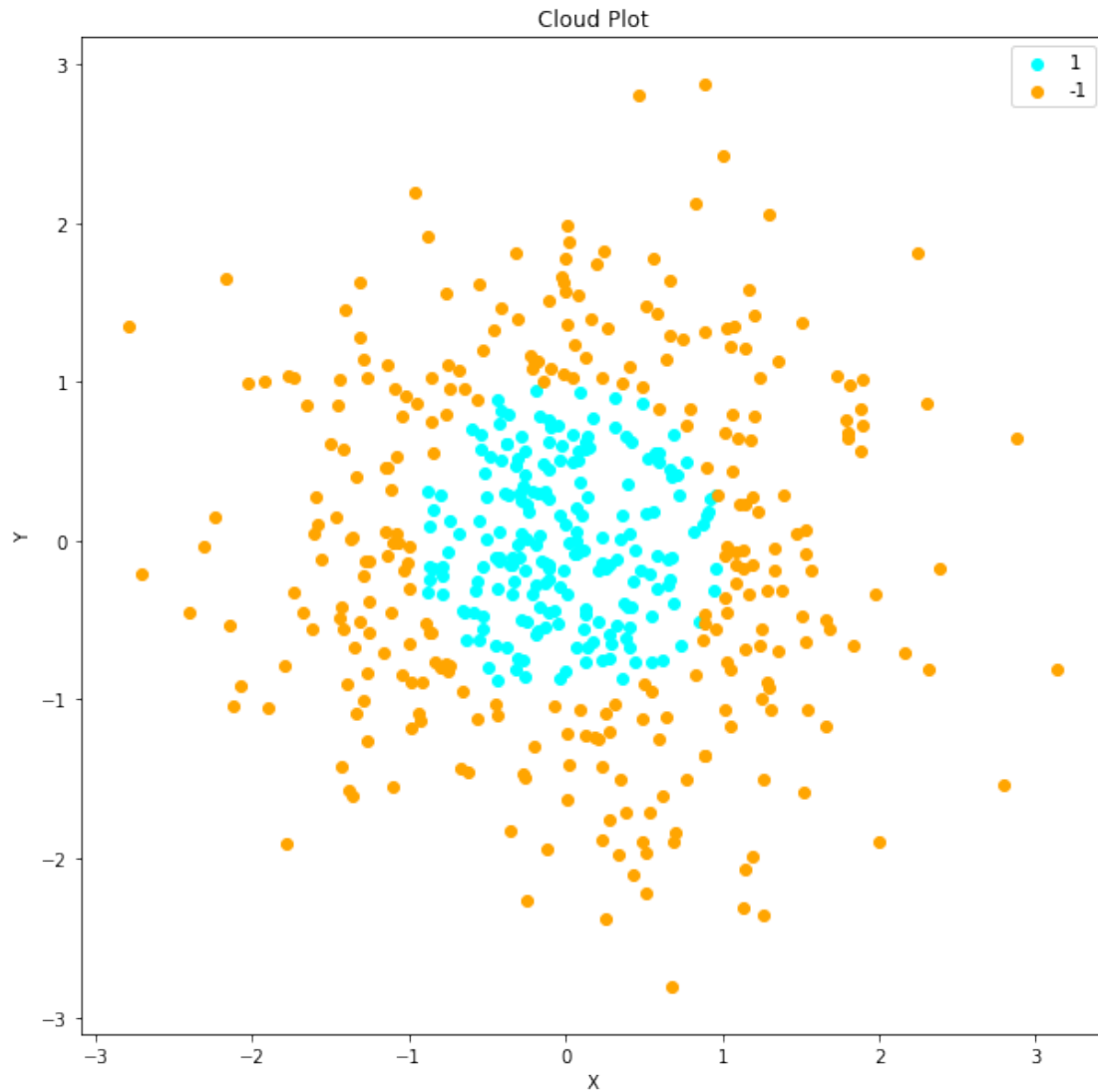
```

1.2 (b)

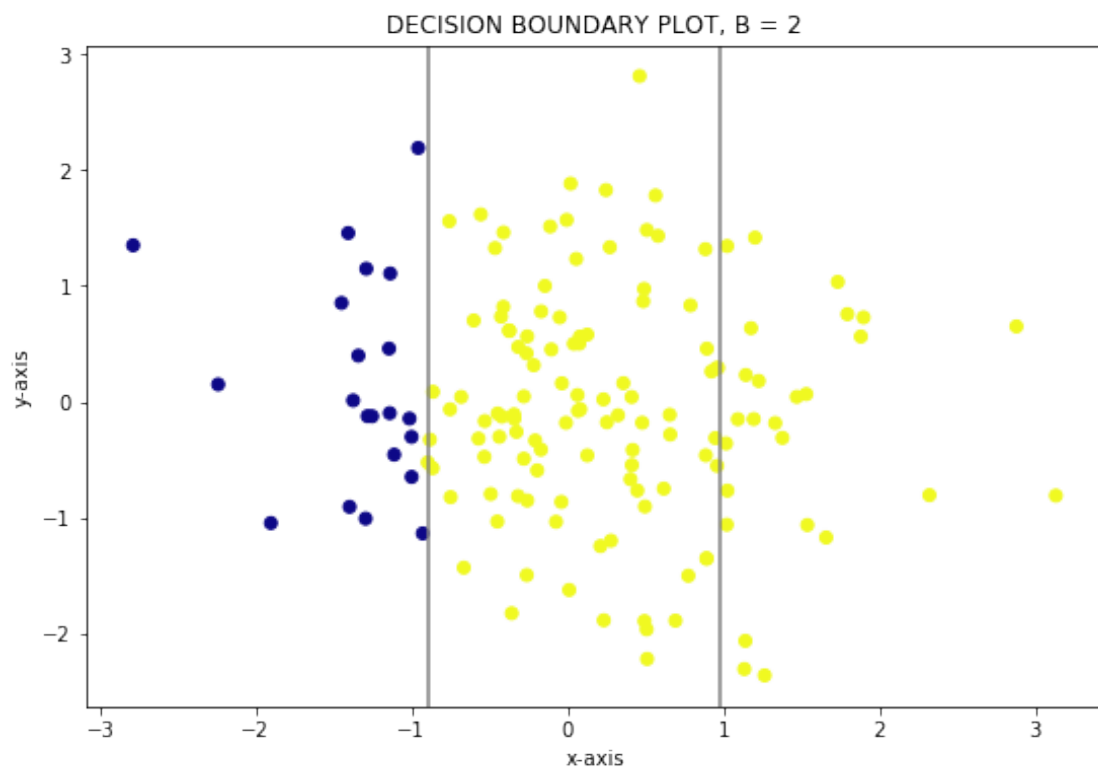
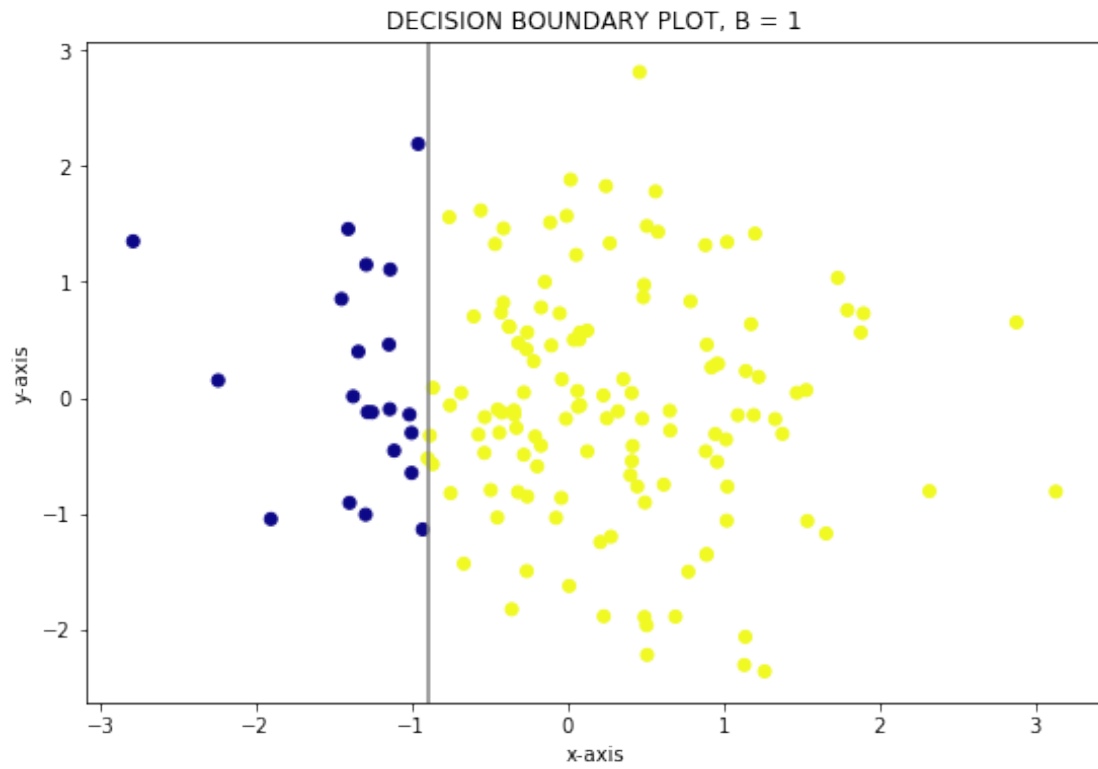
```

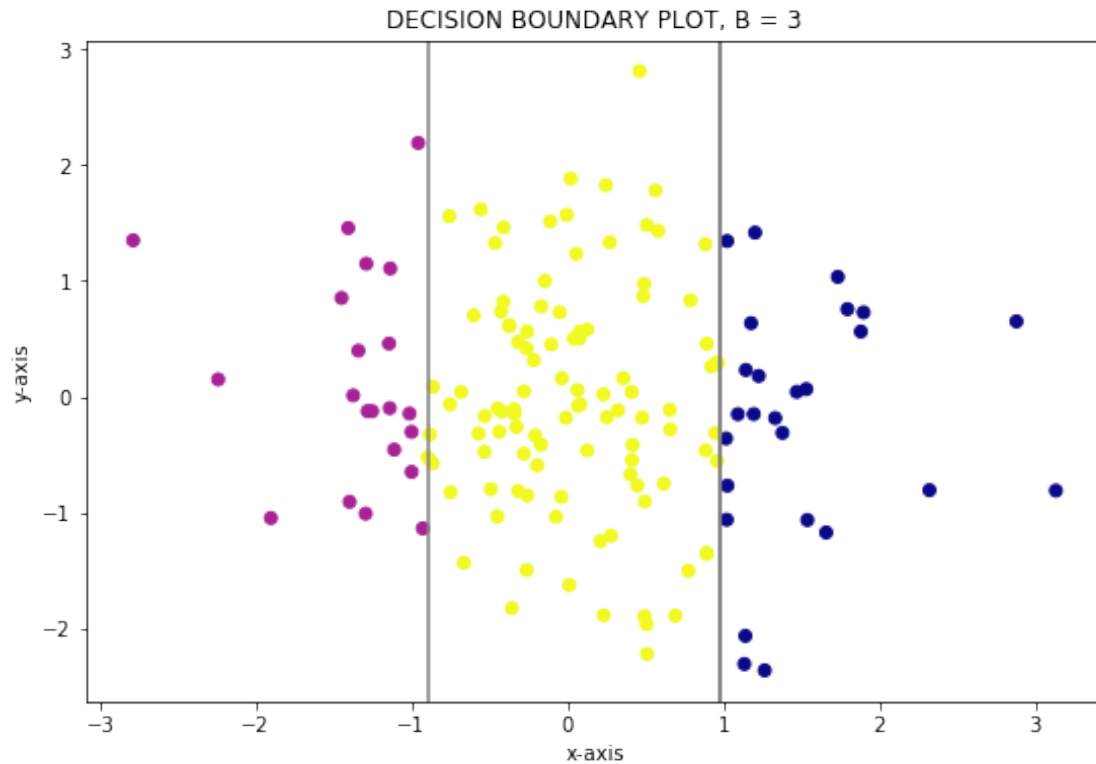
In [16]: n = 500
X, Y = get_circle_data(n)
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size = 0.3)
fig = plt.figure(figsize=(10, 10))
plt.scatter(X[Y == 1,0],X[Y == 1,1], c = 'aqua', label = '1')
plt.scatter(X[Y == -1,0],X[Y == -1,1], c = 'orange', label = '-1')
plt.title('Cloud Plot')
plt.xlabel('X')
plt.ylabel('Y')
plt.legend()
plt.show()

```



```
In [20]: for b in [1,2,3]:
          trees, alphas = my_adaboost(X_train, Y_train, B = b)
          score, preds, test_err = predict_ada(trees, alphas, X_test, Y_test)
          draw_boosted_trees(trees, X_test, score = score)
          plt.title('DECISION BOUNDARY PLOT, B = ' + str(b))
          plt.xlabel('x-axis')
          plt.ylabel('y-axis')
          plt.show()
```



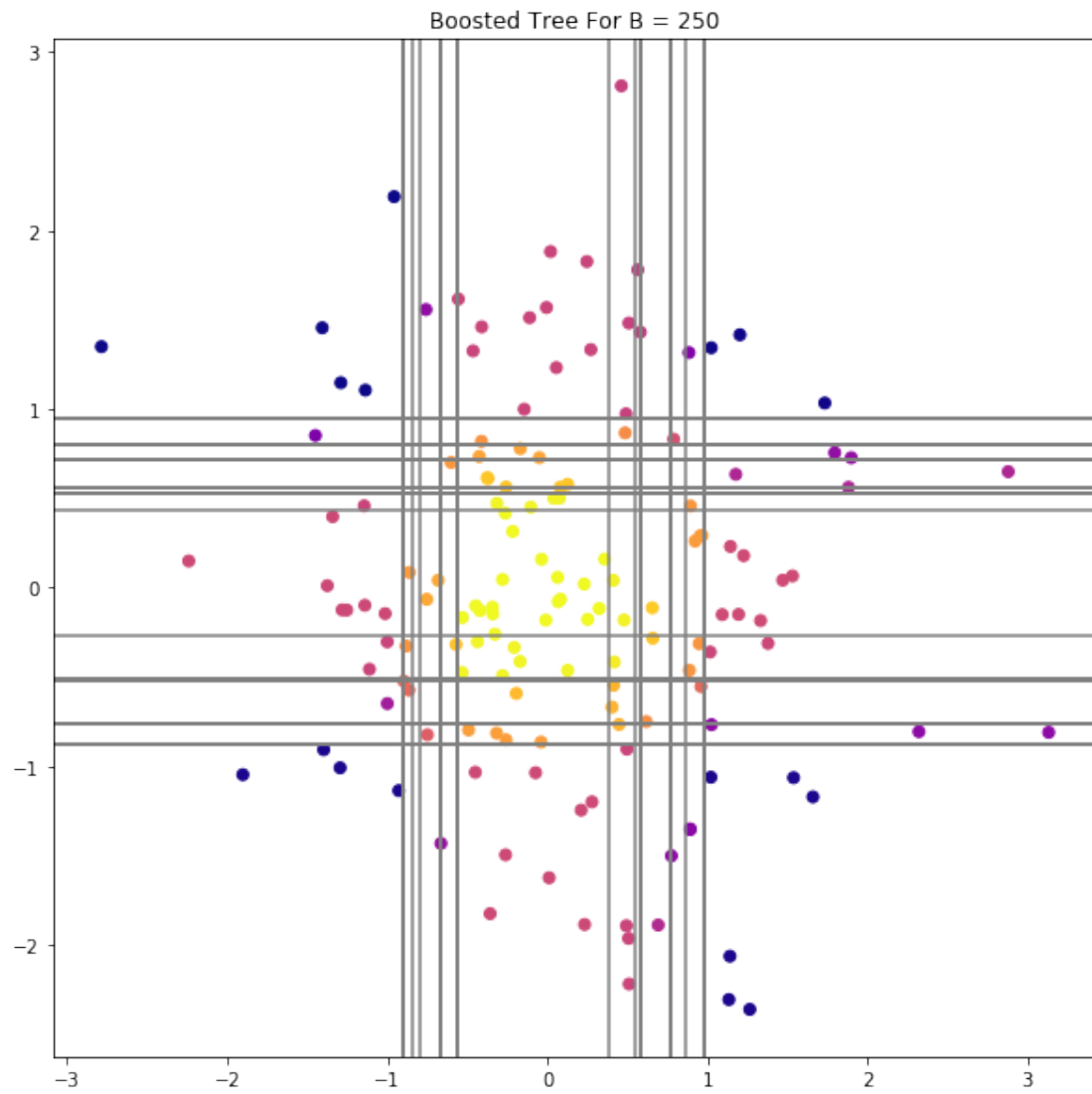


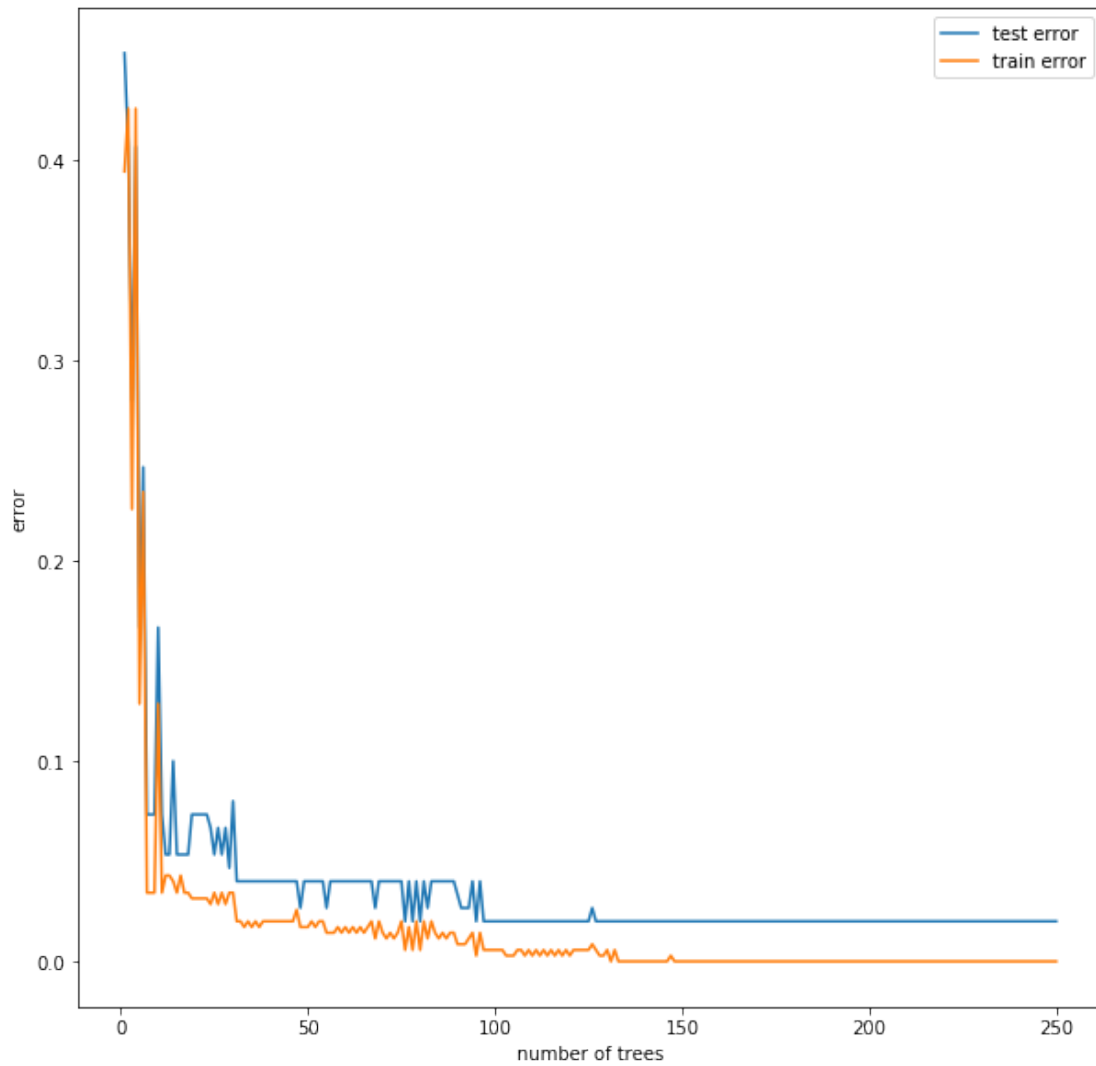
1.3 (c)

```
In [24]: B = 250
trees, alphas = my_adaboost(X_train, Y_train, B = B)
score, preds, test_err = predict_ada(trees, alphas, X_test, Y_test)
score_train, preds_train, test_err_train = predict_ada(trees, alphas, X_train, Y_train)

fig_1 = plt.figure(figsize = (10,10))
draw_boosted_trees(trees, X_test, score = score)
plt.title('Boosted Tree For B = 250')
plt.show()

fig_2 = plt.figure(figsize = (10,10))
plt.plot(np.linspace(1, 250, 250), test_err, label = 'test error')
plt.plot(np.linspace(1, 250, 250), test_err_train, label = 'train error')
plt.xlabel('number of trees')
plt.ylabel('error')
plt.legend()
plt.show()
```





Misclassification rate: 0.02

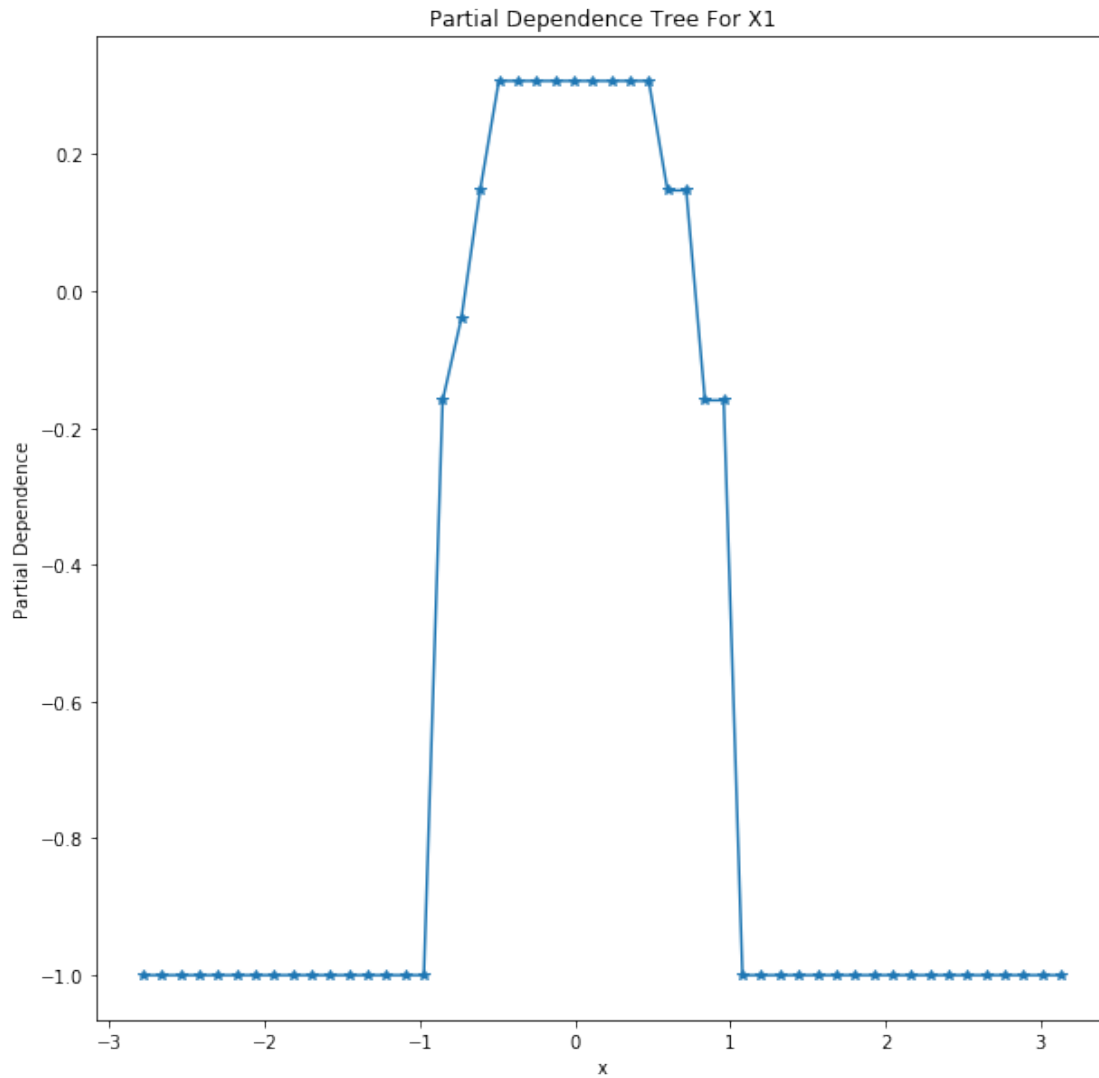
```
In [25]: misclassification_rate = np.mean(preds != Y_test)
         print('Misclassification rate is:', misclassification_rate)
```

Misclassification rate is: 0.02

1.4 (d)

```
In [30]: trees, alphas = my_adaboost(X_train, Y_train, B = 250)
         x, pdep = calculate_partial_dependence_x1(trees, alphas, X_test)
         fig = plt.figure(figsize = (10,10))
```

```
plt.plot(x , pdep, marker = '*')
plt.xlabel('x')
plt.ylabel('Partial Dependence')
plt.title('Partial Dependence Tree For X1')
plt.show()
```



2 Problem 2

2.1 (a)

```
In [90]: X = pd.read_csv('housing_X.csv')
y = pd.read_csv('housing_Y.csv', header = None)
y.columns = ['y']
X_train, X_tmp, y_train, y_tmp = train_test_split(X, y, test_size = 0.4, random_state
```

```

X_test, X_final, y_test, y_final = train_test_split(X_tmp, y_tmp, test_size = 0.5, ra

boost_fit = XGBRegressor(silent = 1).fit(X_train, y_train.values)
yhat = boost_fit.predict(X_test)
MAE = mean_absolute_error(yhat, y_test)
print('MAE Of Boost:', MAE)

```

MAE Of Boost: 37583.45394505444

2.2 (b)

```

In [33]: rf = RandomForestRegressor(n_estimators = 500).fit(X_train, y_train)
yhat_rf= rf.predict(X_test)
MAErf = mean_absolute_error(yhat_rf, y_test)
print('MAE of Random Forest:', MAErf)

```

MAE of Random Forest: 32175.308042084656

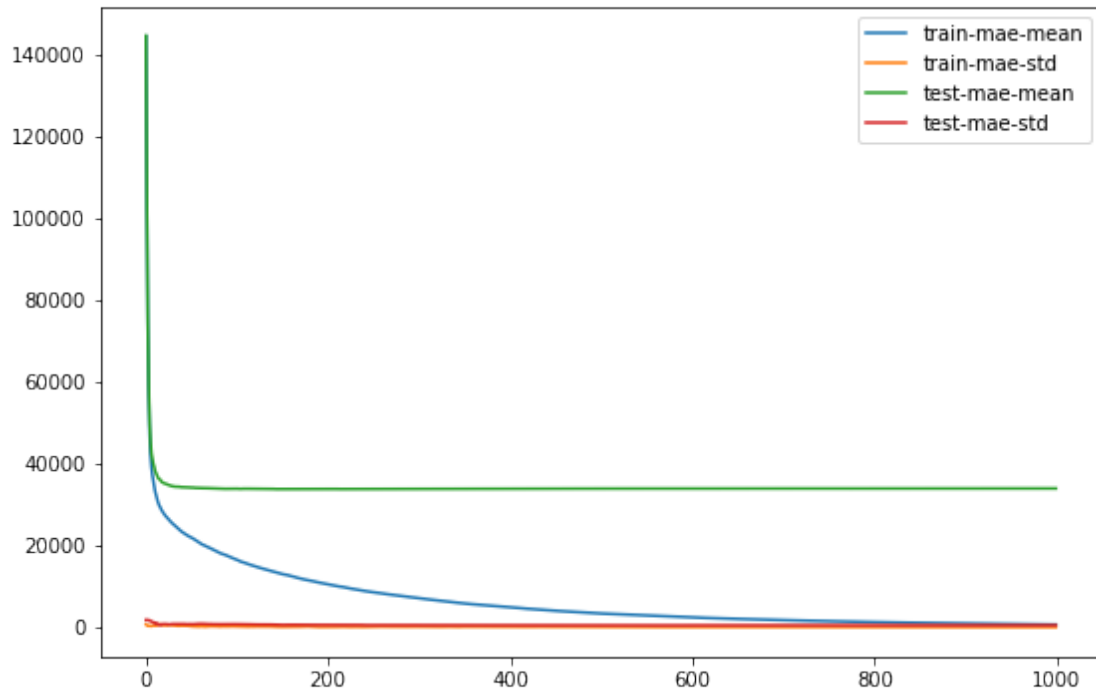
2.3 (c)

As number of trees increases, the expected error decreases, and test error converges to 3500, train error converges to 0.

```

In [36]: train_dm = xgb.DMatrix(X_train.values, y_train)
test_dm = xgb.DMatrix(X_test.values, y_test)
para_dic = {'silent' : 1, 'eval_metric': 'mae'}
cv_times = 1000
cv = xgb.cv(para_dic, train_dm, cv_times)
cv.plot()
plt.show()

```



```
In [44]: m = cv[cv['test-mae-mean']==cv['test-mae-mean'].min()].index.values[0]
# print(cv.iloc[m,:])
print('The minimum number of trees is achived with', m, 'trees')
```

The minimum number of trees is achived with 157 trees

```
In [46]: #What is the MAE on the test set? How does this compare to the previous models?
boost_refit = XGBRegressor(silent = 1, n_estimators = m).fit(X_train, y_train.values)
yhat_refit = boost_refit.predict(X_test)
MAE_refit = mean_absolute_error(yhat_refit, y_test)
print('Original MAE: ', MAE)
print('MAE of Refit: ', MAE_refit)
```

Original MAE: 37583.45394505444

MAE of Refit: 35725.925383647846

The new MAE is 35725.92, which is less than the previous 37583.45 MAE.

2.4 (d)

```
In [48]: para_grid = {'max_depth' : [3, 6, 9, 12], 'min_child_weight': [1, 4, 8, 16]}
boost_fit_grid_cv = GridSearchCV(estimator = XGBRegressor(silent = 1, n_estimators = m))
print(boost_fit_grid_cv.best_estimator_)
yhat_grid_cv = boost_fit_grid_cv.predict(X_test)
MAE_grid_cv = mean_absolute_error(yhat_grid_cv , y_test)
```

```
XGBRegressor(base_score=0.5, booster='gbtree', colsample_bylevel=1,
             colsample_bytree=1, gamma=0, learning_rate=0.1, max_delta_step=0,
             max_depth=9, min_child_weight=8, missing=None, n_estimators=157,
             n_jobs=1, nthread=None, objective='reg:linear', random_state=0,
             reg_alpha=0, reg_lambda=1, scale_pos_weight=1, seed=None, silent=1,
             subsample=1)
```

MAE of Grid CV: 31003.20846300006

```
In [49]: print('MAE of Grid CV:', MAE_grid_cv)
```

MAE of Grid CV: 31003.20846300006

2.5 (e)

```
In [52]: p = {'max_depth': [3,6,9,12],
             'min_child_weight': [1,4,8,16],
             'eta': [0.05, 0.1, 0.3, 0.5, 0.7],
             'gamma': [0, 10, 100, 1000],
             'subsample': [0.1, 0.3, 0.5, 1],
             'colsample_bylevel': [0.5, 0.7, 1]
            }
```

```
boost_fit_rs = RandomizedSearchCV(estimator = XGBRegressor(silent = 1, n_estimators=m
yhat_rs_cv = boost_fit_rs.predict(X_test)
MAE_rs = mean_absolute_error(yhat_rs_cv, y_test)
boost_fit_rs.best_estimator_
```

MAE of Randomized Search: 30451.09448116742

```
Out[52]: XGBRegressor(base_score=0.5, booster='gbtree', colsample_bylevel=0.7,
                    colsample_bytree=1, eta=0, gamma=1000, learning_rate=0.1,
                    max_delta_step=0, max_depth=9, min_child_weight=1, missing=None,
                    n_estimators=157, n_jobs=1, nthread=None, objective='reg:linear',
                    random_state=0, reg_alpha=0, reg_lambda=1, scale_pos_weight=1,
                    seed=None, silent=1, subsample=1)
```

```
In [53]: print('MAE of Randomized Search: ', MAE_rs)
```

MAE of Randomized Search: 30451.09448116742

This model gives MAE of 30451, which is the better than all models above.

2.6 (f)

```
In [94]: p_S = {'max_depth': [3,6,9,12],
               'min_child_weight': [1,4,8,16],
```

```

'gamma': [0, 10, 100, 1000],
'max_delta_step': [0, 1, 2, 4],
'subsample': [0.1, 0.3, 0.5, 1],
'colsample_bylevel': [0.5, 0.7, 1],}
boost_fit_rs_s = RandomizedSearchCV(XGBRegressor(silent = 1),param_distributions = p_
yhat_rs_s = boost_fit_rs_self.predict(X_test)
MAE_rs_s = mean_absolute_error(yhat_rs_s, y_test)
print('MAE of Self Determined:', MAE_rs_s)

```

MAE of Self Determined: 30662.704588290617

In [56]: boost_fit_rs_s.best_estimator_

```

Out[56]: XGBRegressor(base_score=0.5, booster='gbtree', colsample_bylevel=1,
        colsample_bytree=1, gamma=10, learning_rate=0.1, max_delta_step=0,
        max_depth=9, min_child_weight=1, missing=None, n_estimators=100,
        n_jobs=1, nthread=None, objective='reg:linear', random_state=0,
        reg_alpha=0, reg_lambda=1, scale_pos_weight=1, seed=None, silent=1,
        subsample=0.5)

```

2.7 (g)

```

In [71]: yhat_final = boost_fit.predict(X_final)
        MAE_final = mean_absolute_error(yhat_final, y_final)

        yhat_rf_final = rf.predict(X_final)
        MAErf_final = mean_absolute_error(yhat_rf_final, y_final)

        yhat_refit_final = boost_refit.predict(X_final)
        MAE_refit_final = mean_absolute_error(yhat_refit_final, y_final)

        yhat_grid_cv_final = boost_fit_grid_cv.predict(X_final)
        MAE_grid_cv_final = mean_absolute_error(yhat_grid_cv_final , y_final)

        yhat_rs_cv_final = boost_fit_rs.predict(X_final)
        MAE_rs_final = mean_absolute_error(yhat_rs_cv_final, y_final)

        yhat_rs_self_final = boost_fit_rs_s.predict(X_final)
        MAE_rs_self_final = mean_absolute_error(yhat_rs_self_final, y_final)

        listM = ['Boost', 'RF', 'Refit', 'Grid CV', 'RS', 'SD']
        compare = pd.DataFrame(index=['X final'], columns=listM)
        compare.loc['X final'] = [MAE_final, MAErf_final, MAE_refit_final, MAE_grid_cv_final, MAE_
        compare

```

```
Out[71]:
```

	Boost	RF	Refit	Grid CV	RS	SD
X final	37771.4	32239.7	35950.8	30666	30669.1	31174.3

3 Problem 3

3.1 (a)

```
In [103]: train = pd.read_csv('problem3_train.csv')
test = pd.read_csv('problem3_test.csv')
X_train = train.iloc[:, 1:]
y_train = train.iloc[:, 0]
X_test = test.iloc[:, 1:]
y_test = test.iloc[:, 0]
X_train.head()
```

```
Out[103]:
```

	X1	X2	X3	X4	X5	X6	X7	\
0	1.008161	0.019796	1.126327	2.321281	1.105697	0.765064	1.046456	
1	1.822315	-0.445370	0.721193	1.092445	1.527229	-1.154915	-0.492462	
2	-1.111213	-0.654317	0.262071	-0.310766	0.161850	0.460201	-0.516066	
3	-0.242610	1.925626	-0.489186	1.966393	0.014528	-0.099692	0.624639	
4	0.984100	-0.030690	-0.275215	1.726351	0.189390	-0.592996	0.213616	

	X8	X9	X10	...	X41	X42	X43	\
0	-0.644587	0.433121	-1.295707	...	0.039929	-0.568629	-0.238337	
1	0.957552	-1.734727	-0.474635	...	-1.740863	0.065280	-0.070783	
2	-0.204585	-0.476817	0.109984	...	-0.263685	-0.115600	-0.491583	
3	-0.573174	-0.522930	-3.163334	...	-0.822768	-0.597254	-0.093070	
4	-0.634477	0.050807	-0.174530	...	1.650067	0.185604	-1.142097	

	X44	X45	X46	X47	X48	X49	X50
0	1.301445	-1.399641	-1.133262	0.846981	-0.265916	-0.695904	0.442080
1	0.046770	0.330851	0.622000	-0.188007	-1.876895	-0.288879	0.456668
2	-1.018944	2.366216	0.146793	0.690010	-0.768701	0.674229	0.881581
3	-1.157938	-0.871859	1.262199	0.595681	-0.948876	0.179402	0.773642
4	-0.260436	-0.876859	-1.532207	-1.133694	0.967878	0.718648	-2.473946

[5 rows x 50 columns]

```
In [73]: log = LogisticRegression().fit(X_train,y_train)
loghat_train = log.predict(X_train)
loghat_test = log.predict(X_test)
misclassification_train = np.mean(loghat_train != y_train)
misclassification_test = np.mean(loghat_test != y_test)
print('Train misclassification rate:', misclassification_train)
print('Test misclassification rate:', misclassification_test)
```

The train misclassification rate is 0.14333333333333334
The test misclassification rate is 0.2365

3.2 (b)

```
Train misclassification rate is 0.17
Test misclassification rate is 0.216
```

There is an improvement of misclassification rate in the test.

```
In [97]: features = np.array(train.columns[np.where(np.abs(logcv.coef_[0]) > 0)])
len(features)
```

Out [97]: 30

```
In [100]: # df = pd.DataFrame(log_cv.coef_[0], index = X_train.columns[1:], columns=['beta'])
# print('These are the variables used in the model')
# df.loc[df.beta.abs()>0,:]
```

```
features = np.array(train.columns[np.where(np.abs(logcv.coef_[0]) > 0)])
listM = np.arange(len(features))
compare = pd.DataFrame(index=['Features'], columns=listM)
compare.loc['Features'] = features
compare
```

```
Out[100]:
```

	0	1	2	3	4	5	6	7	8	9	...	20	21	22	\
Features	X2	X3	X6	X7	X8	X12	X14	X15	X17	X19	...	X32	X33	X34	

	23	24	25	26	27	28	29
Features	X36	X39	X42	X44	X46	X47	X49

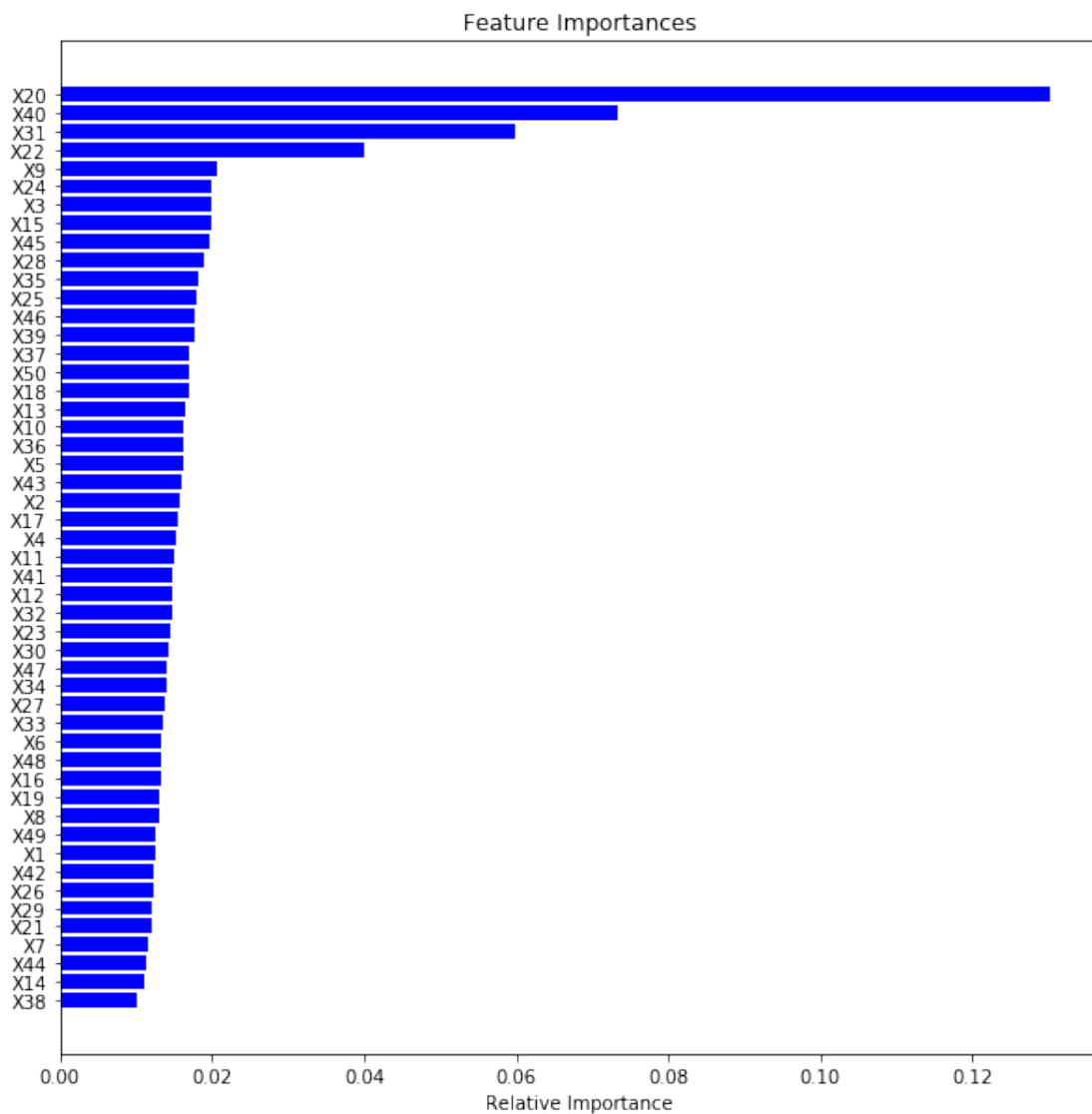
[1 rows x 30 columns]

3.3 (c)

```
In [106]: fit_rf = RandomForestClassifier(n_estimators = 500, oob_score = True).fit(X_train, y_train)
train_error = np.mean(fit_rf.predict(X_train)!=y_train)
test_error = np.mean(fit_rf.predict(X_test)!=y_test)
print('Out-of-bag error:', 1 - fit_rf.oob_score_)
print('Train Error: ', train_error)
print('Test Error: ', test_error)
```

Out-of-bag error: 0.24
Train Error: 0.0
Test Error: 0.2231

```
In [109]: # explore the random forest to figure out how it is making its predictions, and how y
features = pd.get_dummies(X_train).columns
importances = fit_rf.feature_importances_
indices = np.argsort(importances)
plt.figure(figsize = (10,10))
plt.title('Feature Importances')
plt.barh(range(len(indices)), importances[indices], color='b', align='center')
plt.yticks(range(len(indices)), [features[i] for i in indices])
plt.xlabel('Relative Importance')
plt.show()
```



Using random forest, we explore the importance of different variables. It is clear that X20, X40 and X31 are the most important variables in our model.

3.4 (d)

```
In [116]: log_lasso = LogisticGAM().gridsearch(X_train[['X20', 'X40', 'X31']].values, y_train)
ypred = log_lasso.predict(X_train[['X20', 'X40', 'X31']])
ypred_test = log_lasso.predict(X_test[['X20', 'X40', 'X31']])
train_error = np.mean( ypred != y_train)
test_error = np.mean( ypred_test != y_test)
print('Train Error:', train_error)
print('Test Error:', test_error)

# log_lasso = LogisticGAM().gridsearch(X_train[['X20', 'X40']].values, y_train)
# ypred = log_lasso.predict(X_train[['X20', 'X40']])
# ypred_test = log_lasso.predict(X_test[['X20', 'X40']])
# train_error = np.mean( ypred != y_train)
# test_error = np.mean( ypred_test != y_test)
# print('Train Error:', train_error)
# print('Test Error:', test_error)

# log_lasso = LogisticGAM().gridsearch(X_train[['X20', 'X40', 'X31', 'X22']].values, y_
# ypred = log_lasso.predict(X_train[['X20', 'X40', 'X31', 'X22']])
# ypred_test = log_lasso.predict(X_test[['X20', 'X40', 'X31', 'X22']])
# train_error = np.mean( ypred != y_train)
# test_error = np.mean( ypred_test != y_test)
# print('Train Error:', train_error)
# print('Test Error:', test_error)
```

100% (11 of 11) |#####| Elapsed Time: 0:00:00 Time: 0:00:00

Train Error: 0.08666666666666667

Test Error: 0.1313

The test error is 0.1313, very close to 0.12, using predictors X20, X40 and X31.